Problem 1 5-Stage Pipeline RAW Hazards & Hardware Forwarding

Part A

We have to insert 3 NOPs before sub, then 0 additional NOPs before and and 0 before or. The total is 3 stalls. All three instructions read x1, but delaying sub also delays the later consumers; so, we should not count the delays separately.

The add instruction produces its result at the end of EX in CC3 and writes x1 in the second half of WB in CC5. An ID read in the first half of CC5 still sees the old value. Therefore, sub must read x1 in CC6 or later. Its unstalled ID cycle would be CC3, so it must be delayed by three cycles.

$$Required\ delay\ =\ 6\ -\ 3\ =\ 3\ cycles$$

A valid space-time schedule uses three explicit NOP instructions. A dash marks a cycle in which that instruction is absent. NOP rows are shown to make the inserted spacing explicit.

Instruction	1	2	3	4	5	6	7	8	9	10	11
add	IF	ID	EX	MEM	WB	–	–	–	–	–	–
NOP 1	–	IF	ID	EX	MEM	WB	–	–	–	–	–
NOP 2	–	–	IF	ID	EX	MEM	WB	–	–	–	–
NOP 3	–	–	–	IF	ID	EX	MEM	WB	–	–	–
sub	–	–	–	–	IF	ID	EX	MEM	WB	–	–
and	–	–	–	–	–	IF	ID	EX	MEM	WB	–
or	–	–	–	–	–	–	IF	ID	EX	MEM	WB

Part B With hardware forwarding

The answer depends on whether “full hardware forwarding” also includes a WB-to-ID register-read bypass. With that bypass, the usual answer is **0 stalls**. If Part A’s read-first/write-second timing remains in force and only the two expressly listed EX forwarding paths exist, the complete sequence **requires 1 stall before or.**

Consumer	EX cycle without stalls	Source supplying x1	ALU input
sub x4, x1, x5	CC4	EX/MEM.ALUResult from add	A (rs1)
and x6, x1, x7	CC5	MEM/WB write-back value from add	A (rs1)

or x8, x1, x9	CC6	ID/EX.ReadData1, made correct by WB-to-ID bypass in CC5	A (rs1)

The second ALU operand comes from ID/EX.ReadData2: x5 for sub, x7 for and, and x9 for or. No forwarding to ALU input B is needed because x1 is rs1 in every consumer. For add, the MEM/WB write-back value is its ALU result, rather than loaded memory data.

Part C Replacing add with lw

The load-use dependency between lw and the sub right after it costs one stall. If "full forwarding" includes a WB-to-ID bypass, that single stall is the only one in the four-instruction sequence. If only the listed EX paths exist, and registers are read in the first half of a cycle and written in the second, the sequence needs an additional stall tied to WB-to-ID, for 2 stalls in total.

The load computes its address in EX during CC3, but the actual x1 value doesn't exist until the end of MEM in CC4. With no stall, sub would need x1 at the start of its EX stage in that same cycle. Forwarding can't supply a value that hasn't been produced yet, and at that moment the EX/MEM register holds only the load's address, not the data.

If sub is held back one cycle, its EX stage moves to CC5. By then the memory data selected for write-back is sitting in MEM/WB, and it gets forwarded into the ALU's input A of sub. Input B still carries x5.

Schedule when WB to ID bypass is included

Instruction	IF	ID	EX	MEM	WB
lw x1, 0(x2)	1	2	3	4	5
sub x4, x1, x5	2	3–4	5	6	7
and x6, x1, x7	3–4	5	6	7	8
or x8, x1, x9	5	6	7	8	9

Ranges denote a stage held by the interlock. An EX bubble appears in CC4. In CC5, and gets x1 through the WB-to-ID bypass; or reads the already updated register file in CC6. Final completion is CC9.

Schedule with only the two listed EX bypasses

Instruction	IF	ID	EX	MEM	WB
lw x1, 0(x2)	1	2	3	4	5
sub x4, x1, x5	2	3–4	5	6	7
and x6, x1, x7	3–4	5–6	7	8	9
or x8, x1, x9	5–6	7	8	9	10

After the load-use stall, and reaches ID in CC5, before lw writes x1 late in that cycle. It must read again in CC6 and enter EX in CC7. There are two EX bubbles, in CC4 and CC6. Only the first is the intrinsic load-use delay; the second is caused by the missing WB-to-ID bypass.

Problem 2 Branch prediction and CPI

Assume the instruction count and clock period stay unchanged, correctly predicted branches have no penalty, every branch misprediction costs 2 cycles, and the stated hazard penalties add without overlap. The 2-cycle misprediction model is necessary to use the supplied overall prediction accuracy in Part B.

Part A Static predict not taken

The ideal CPI already includes normal execution of every instruction. Adding only the expected stall cycles per committed instruction, Twenty percent of instructions are loads, and twenty percent of those loads cause one stall.

$$Load\ stall\ CPI\ =\ 0.20 \times 0.20 \times 1\ =\ 0.04$$

With predict-not-taken, each taken conditional branch is mispredicted. The misprediction probability among branches is therefore 0.70.

$$Branch\ stall\ CPI\ =\ 0.20 \times 0.70 \times 2\ =\ 0.28$$

$$CPI\ static\ =\ 1.00\ +\ 0.04\ +\ 0.28\ =\ 1.32$$

Part B Dynamic two bit predictor

A 90% prediction accuracy gives a 10% misprediction rate among branches. We can not multiply that rate by the taken fraction again: the accuracy already covers all branch outcomes. The load contribution is unchanged.

$$Branch\ stall\ CPI\ =\ 0.20 \times (1\ -\ 0.90) \times 2\ =\ 0.04$$

$$CPI\ dynamic\ =\ 1.00\ +\ 0.04\ +\ 0.04\ =\ 1.08$$

Part C Overall performance speedup

Execution time equals instruction count multiplied by CPI and clock period. With the same workload and clock period, those common factors cancel in the old-time/new-time ratio.

$$Speedup\ =\ CPI\ static\ /\ CPI\ dynamic\ =\ 1.32\ /\ 1.08\ =\ 1.2222\ldots$$

The upgrade yields approximately 1.222× speedup, or 22.22% higher performance. Execution time falls by (1 − 1.08/1.32) × 100% = 18.18%.

Problem 3 Clock timing and stage imbalance

Part A Minimum period and maximum frequency

The slowest combinational stage determines the pipeline clock period. Adding the specified 20 ps register overhead once per clock period,

$$T\,clock \;=\; max(160,\ 120,\ 190,\ 200,\ 110) \;+\; 20 \;=\; 220\,ps$$

$$f\,max \;=\; 1\ /\ (220 \times 10^{-12}\,s) \;=\; 4.54545 \times 10^{9}\,Hz \approx 4.545\,GHz$$

MEM is the limiting stage at 200 ps. The 220 ps clock accommodates its logic delay plus the register setup and clock-to-Q overhead.

Part B Ideal speedup over a single cycle implementation

$$T\,single \;=\; 160 \;+\; 120 \;+\; 190 \;+\; 200 \;+\; 110 \;=\; 780\,ps$$

$$f\,single \;=\; 1\ /\ (780 \times 10^{-12}\,s) \approx 1.282\,GHz$$

For a long instruction stream with ideal CPI = 1 in both designs, throughput is proportional to clock frequency. Using the question's single-cycle period without adding extra overhead:

$$Speedup \;=\; 780\ /\ 220 \;=\; 3.54545\ldots \approx 3.545\times$$

This is a steady-state throughput speedup. It is not the latency speedup of one instruction: one pipelined instruction spans 5 × 220 = 1100 ps, versus 780 ps in the single-cycle implementation. For N independent instructions, including pipeline fill and drain, the time ratio is 780N / [220(N + 4)], approaching 3.545× as N becomes large.

Part C Why the speedup is below five times

The first loss comes from unbalanced stages. If the 780 ps of total combinational logic were split evenly across five stages, each would need only 156 ps. In reality the MEM stage needs 200 ps, and since every stage shares one clock, the quicker stages sit idle until it finishes. Even if registers added no delay at all, this alone caps the speedup at 780/200 = 3.90×.

The second loss is the pipeline registers themselves. Setup time plus clock-to-Q delay stretch the critical period from 200 ps to 220 ps, which drops the speedup from 3.90× to about 3.545×. As a point of comparison, a perfectly balanced design carrying the same 20 ps register cost would have a 176 ps clock and reach 780/176 ≈ 4.432×, which is still short of the ideal 5×.

Other real-world costs may also apply, such as delays through forwarding muxes and control logic, bypass wiring, and clock skew, unless those are already folded into the stage timings. Data hazards, branch flushes, structural conflicts, and the time spent filling and draining the pipeline would lower actual performance even more. They don't account for the ideal-CPI figure above, though, because that calculation leaves out stalls and assumes the pipeline is in steady state.

Part-B Hands-on experience Gem5 simulator

Step-1

Both commands have been executed successfully

```
(gem5-env) shahriarahi@Shahriar:~/gem5$ test ! -e m5out-step1-timing-screenshot || { echo "Output directory already exists"; exit 1; }

LD_LIBRARY_PATH="$CONDA_PREFIX/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}" \
  timeout 110s build/RISCV/gem5.opt --outdir=m5out-step1-timing-screenshot \
  configs/deprecated/example/se.py --cpu-type=TimingSimpleCPU --caches \
  -c tests/test-progs/hello/bin/riscv/linux/hello
echo "TimingSimpleCPU exit code: $?"
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Sep  6 2026 18:58:20
gem5 started Sep 27 2026 12:29:58
gem5 executing on Shahriar, pid 14444
command line: build/RISCV/gem5.opt --outdir=m5out-step1-timing-screenshot configs/deprecated/example/se.py --cpu-type=TimingSimpleCPU --ca
ches -c tests/test-progs/hello/bin/riscv/linux/hello

warn: The se.py script is deprecated. It will be removed in future releases of  gem5.
Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/arch/riscv/isa.cc:321: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group.
Legacy stat is deprecated.
system.remote_gdb: Listening for connections on port 7000
**** REAL SIMULATION ****
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
Hello world!
Exiting @ tick 30457000 because exiting with last active thread context
TimingSimpleCPU exit code: 0
(gem5-env) shahriarahi@Shahriar:~/gem5$ test ! -e m5out-step1-o3-screenshot || { echo "Output directory already exists"; exit 1; }

LD_LIBRARY_PATH="$CONDA_PREFIX/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}" \
  timeout 110s build/RISCV/gem5.opt --outdir=m5out-step1-o3-screenshot \
  configs/deprecated/example/se.py --cpu-type=O3CPU --caches \
  -c tests/test-progs/hello/bin/riscv/linux/hello
echo "O3CPU exit code: $?"
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Sep  6 2026 18:58:20
gem5 started Sep 27 2026 12:30:12
gem5 executing on Shahriar, pid 14555
command line: build/RISCV/gem5.opt --outdir=m5out-step1-o3-screenshot configs/deprecated/example/se.py --cpu-type=O3CPU --caches -c tests/
test-progs/hello/bin/riscv/linux/hello

warn: The se.py script is deprecated. It will be removed in future releases of  gem5.
Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/arch/riscv/isa.cc:321: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group.
Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
system.remote_gdb: Listening for connections on port 7000
**** REAL SIMULATION ****
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
Hello world!
Exiting @ tick 14677500 because exiting with last active thread context
O3CPU exit code: 0
(gem5-env) shahriarahi@Shahriar:~/gem5$
```

Step 2: Hardware Telemetry Extraction Table

Gem5 stat key	TimingSimpleCPU(In-order)	O3CPU(Out-of-Order)
sim_ticks	30457000	14677500
System.cpu.numCycles	60914	29357
System.cpu.ipc	0.090587	0.187962
System.cpu.dcache.overall_misses	135	524

```
(gem5-env) shahriarahi@Shahriar:~/gem5$ for model in timing o3; do
  stats="m5out-step1-$model-screenshot/stats.txt"
  echo "=== $model ==="
  awk '
    $1 == "sim_ticks" ||
    $1 == "simTicks" ||
    $1 == "system.cpu.numCycles" ||
    $1 == "system.cpu.ipc" ||
    $1 == "system.cpu.cpi" ||
    $1 == "system.cpu.dcache.overallMisses::total" {
      print $1, $2
    }
  ' "$stats"
done
=== timing ===
simTicks 30457000
system.cpu.numCycles 60914
system.cpu.cpi 11.039145
system.cpu.ipc 0.090587
system.cpu.dcache.overallMisses::total 135
=== o3 ===
simTicks 14677500
system.cpu.numCycles 29357
system.cpu.cpi 5.320225
system.cpu.ipc 0.187962
system.cpu.dcache.overallMisses::total 524
(gem5-env) shahriarahi@Shahriar:~/gem5$
```

Step 3: Reflection

On the same RISC-V hello binary, O3CPU finished in 14,677,500 simulated ticks and 29,357 cycles (IPC 0.187962, CPI 5.320225). TimingSimpleCPU needed 30,457,000 ticks and 60,914 cycles (IPC 0.090587, CPI 11.039145). A likely reason for the roughly 2.075× IPC advantage is that the out-of-order core can issue independent instructions that are ready while other instructions are still stalled. That said, these aggregate counters can't isolate that effect. O3CPU also logged more L1 data-cache misses than TimingSimpleCPU (524 versus 135), so a lower miss count can't be what makes its runtime shorter.